PROJECT REPORT

Language Bridge: Native-Language Vocabulary Suggestions and Sarvam Speech Recognition

Project: Lexa — AI voice conversation coach

Scope: English-alternatives report for Indic languages; speech-to-text routing for Kannada, Tamil and Telugu

Stack: FastAPI (Python 3.12), React 19 / TanStack Start, Groq, Sarvam, Deepgram

Status: Implemented, tested (442 passing tests), running locally

Executive summary

The application supports a **language bridge** mode in which a learner speaks Hindi, Kannada, Tamil or Telugu, is answered in simple English, and at the end receives a report of English vocabulary they could use next time.

This work addressed two weaknesses in that mode:

1. **The vocabulary report was suggesting English-to-English upgrades.** It would take an English word already present in a Hinglish transcript and propose a more formal synonym, instead of translating the learner's own language. The prompt was rewritten and a deterministic validation layer was added so that every reported item has a genuine native-script left-hand side and a genuinely English right-hand side. Per-language loanword and common-word data was added for all four languages.
2. **Kannada, Tamil and Telugu transcription relied on Whisper, which is not specialised for Indic languages.** Those three languages are now routed to **Sarvam** `saaras:v4`, while Hindi and English remain on Whisper through Groq. The reason is distribution match: Whisper's training corpus is dominated by English and major European languages, while Sarvam's `saaras` family is Indic-first, takes the language as an explicit BCP-47 input, and exposes an output mode for code-mixed speech. The trade is higher latency, no word timestamps and a second vendor — none of which affects the bridge feature today. Routing is transparent to the conversation service and degrades safely to Whisper if the Sarvam key is absent.

No database migration or API schema change was required. Existing stored reports are replaced when the analysis is re-run.

---

1. Introduction

1.1 Product context

Lexa is a spoken-English coach. A learner talks to an AI interviewer, and the system analyses grammar, vocabulary, fluency and pronunciation. The **language bridge** is an adjacent feature for learners who are not yet comfortable holding a whole conversation in English: they speak their own language, the assistant replies in simple English, and the session ends with a vocabulary report.

Four spoken languages are supported:

Code	Language	Script	Unicode block
`hi`	Hindi	Devanagari	U+0900–U+097F
`kn`	Kannada	Kannada	U+0C80–U+0CFF
`ta`	Tamil	Tamil	U+0B80–U+0BFF
`te`	Telugu	Telugu	U+0C00–U+0C7F

1.2 Objectives

1. Make the English-alternatives report translate **native-language words and phrases into English**, not rewrite English that the learner already used.
2. Apply the same behaviour consistently to all four bridge languages, using shared logic and language-specific data.
3. Improve transcription for Kannada, Tamil and Telugu by using a speech-to-text engine built for Indian languages.
4. Keep the change operationally safe: no forced migration, no hard failure when a provider is unconfigured, and automated tests for every new invariant.

---

2. Background and problem analysis

2.1 Code-mixing and loanwords

Everyday speech in all four languages mixes in English words: *office*, *meeting*, *phone*, *time*, *problem*. Recognition may write those words in the native script (मीटिंग, ఆఫీస్, மீட்டிங், ಮೀಟಿಂಗ್) or in Latin letters, depending on the engine. A vocabulary report is only useful if it distinguishes:

- a **native word** the learner may not know in English (`काम`, `ಕೆಲಸ`, `வேலை`, `పని`), which is worth translating; from
- an **English word** the learner already knows, which is not.

2.2 Root cause of the English-to-English suggestions

Two independent causes were found.

Prompt drift. The system prompt asked for *"a single word whose English equivalent is a less common, longer or more formal word"*. That is a synonym-upgrade instruction. Given a Hinglish transcript it would propose, for example, `office → workplace` rather than translating a Hindi word.

No validation of the left-hand side. The post-processing function `clean()` checked that the proposed **English** side looked English (`_is_english`), that the item appeared in the transcript, and that it was not an exact duplicate. It never checked that the **original** was in the learner's language. A Latin-script English word therefore passed every check.

2.3 Speech recognition differences

Whisper is a strong multilingual model and excellent on English and Hindi, but its accuracy on Kannada, Tamil and Telugu is lower, with more spelling noise. The application already pinned the language per turn to avoid Whisper's auto-detection errors (it had labelled Kannada as Tamil and Telugu as Hindi). Pinning fixed the wrong-language failures but did not close the accuracy gap on the three Dravidian languages.

Sarvam's `saaras` family is trained specifically for Indian languages and was selected to close that gap.

---

3. System architecture

The relevant slice of the backend is provider-oriented:

```
ConversationService
  ├── SpeechToTextProvider        (interface)
  │     ├── GroqSpeechToTextProvider      (Whisper via Groq)
  │     ├── SarvamSpeechToTextProvider    (saaras via Sarvam)
  │     ├── LanguageRoutingSpeechToTextProvider
  │     └── MockSpeechToTextProvider      (tests)
  ├── LLMProvider                 (Groq)
  └── TextToSpeechProvider        (Deepgram, with fallback)

EnglishAlternativesService
  └── EnglishAlternativesProvider
        ├── GroqEnglishAlternativesProvider
        └── MockEnglishAlternativesProvider
```

The conversation service only knows that it is transcribing audio for a given language. It does not know which vendor serves that language. The choice is made in the provider registry and the routing adapter.

Data flow for one turn:

```
audio upload
  → ffmpeg normalisation (16 kHz mono WAV)
  → SpeechToTextProvider.transcribe(..., language=session.spoken_language)
  → transcript stored on the turn
  → LLM reply in simple English
  → TTS audio returned to the client
```

At the end of the session, the alternatives endpoint sends the learner's utterances to the configured LLM provider, applies deterministic validation, ranks the survivors, and stores the report.

4. Part A — English alternatives for native vocabulary

4.1 Design principles

The implementation separates **judgement** from **invariants**:

- The language model decides **what is worth learning**.
- Code decides **what is allowed to be reported**.

This matters because prompt instructions are probabilistic. A model can ignore them, especially on noisy recognition output. The invariants are therefore enforced in Python after the model responds.

4.2 Prompt redesign

The prompt now:

- asks for **common, everyday words** a learner is likely to reach for, and for phrases where the natural English is not word-for-word;
- bans Latin-script `original` values and mixed-script spans explicitly;
- names concrete English loanwords in the target script, so the exclusion is not left to guesswork;
- requires the `original` to be copied **exactly** from the transcript, without correction or transliteration;
- requires the shortest span that carries the meaning (one word, or a phrase of two to five words);
- requires the `english` field to be a reusable word or chunk, not a full sentence and not a fancier synonym of an English word;
- caps the output at 12 items and allows an empty list.

A representative excerpt:

> **LEAVE OUT** - Anything written in Latin letters. Every `original` must be *{script}* script only — never English letters, and never a span that mixes the two scripts. - English words written in *{script}*. Everyday speech is full of them and the learner already knows them. Examples in *{script}* script: …
>
> **RULES** - `original` is copied exactly as written in their sentence … Never rewrite, correct or transliterate it, even if it looks misspelled. - `original` is the SHORTEST span that carries the expression: one word, or a phrase of two to five words. - `english` is the natural, reusable English for that span, in Latin script only … Never answer with a fancier synonym of an English word: the left side must be *{name}*, the right side English.

The prompt is generated per language (`_prompt_for(language)`), using the language's name, script, speech notes and a sample of its loanword list.

4.3 Deterministic filtering pipeline

Every proposed item passes the following checks, in order. A failure at any step drops the item.

Step	Check	Purpose
1	`english` is non-empty and written in Latin script	Reject malformed or native-script right-hand sides
2	`original` matches a real transcribed span (fuzzy ratio ≥ 0.8, ≤ 6 tokens)	Reject invented quotations and clause-length spans
3	`original` contains **no** Latin letters and **at least one** letter from the language's Unicode block	Reject English words and mixed-script spans
4	`original` is not an all-loanword span	Reject English words written in the native script
5	Not a duplicate of an earlier item; `original` $\neq$ `english`	Keep the report short and non-redundant
6	Rank common words first, then cap at 12	Prefer reusable vocabulary

The matcher in step 2 slides a window of the quoted length (plus or minus one word, because recognition sometimes splits or joins words) across every utterance and keeps the closest match. The reported `original` is then replaced by the **transcribed text** it matched, so the learner never sees a tidied or invented spelling.

4.4 Script validation

The script check is language-specific:

```
def _in_script(text: str, language: SpokenLanguage) -> bool:
    low, high = language.script_range
    found = False
    for char in text:
        if not char.isalpha():
            continue
        code = ord(char)
        if 0x0041 <= code <= 0x024F:   # any Latin letter disqualifies
            return False
        if low <= code <= high:
            found = True
    return found
```

This catches the main failure mode (`office → workplace`), but not an English word written in Devanagari, Kannada, Tamil or Telugu letters. That requires lexical knowledge.

4.5 Loanword filtering

Each language has a curated set of high-frequency English loanwords in its own script. Function words (postpositions, particles, pronouns) are ignored when deciding whether a span is "all loanword", so a loanword with a suffix is still recognised:

- `मीटिंग में` → content word `मीटिंग` → dropped
- `ಮೀಟಿಂಗ್ ನಲ್ಲಿ` → content word `ಮೀಟಿಂಗ್` → dropped

- `మీటింగ్ లో` → content word `మీటింగ్` → dropped

A span that also contains a genuine native word survives, because there is still something to translate.

```
def _is_loanword(text: str, language: SpokenLanguage) -> bool:
    loanwords = {_normalise(word) for word in language.loanwords}
    function_words = {_normalise(word) for word in language.function_words}
    content = [
        normalised
        for token in text.split()
        if (normalised := _normalise(token))
        and normalised not in function_words
    ]
    return bool(content) and all(token in loanwords for token in content)
```

4.6 Language-specific knowledge

All four languages now carry four pieces of data:

Language	Loanwords	Common words	Function words
Hindi	77	97	42
Kannada	71	84	32
Tamil	70	84	30
Telugu	71	84	30

- **Loanwords** cover everyday technology, work, education, travel, health and media vocabulary.
- **Common words** are everyday native words (work, money, time, help, need, quickly, difficult, family) used for ranking, never for hiding.
- **Function words** are grammar-only tokens used by the loanword filter.

The word lists are deliberately high-frequency rather than exhaustive. They are a safety net, not a dictionary.

4.7 Ranking

The model cannot know which of its items a learner will reuse most. A per-language common-word list therefore orders the survivors before the 12-item cap:

```
def sort_key(pair):
    index, item = pair
    words = {_normalise(word) for word in item.original.split()}
    common = bool(words & common_words)
    return (0 if common else 1, index)
```

Common words come first; ties keep the provider's order. Ranking never removes an item by itself — it only decides what survives the cap.

4.8 Observed results

The following transcripts were supplied directly to the Groq provider to exercise the prompt and filters. They are representative sentences, not recordings.

Hindi

```
Input:
  कल मीटिंग में मेरा काम अधूरा रह गया, थोड़ी मदद चाहिए।
  वह बहुत जल्दी चला गया।
  मुझे इस बात का फ़र्क समझ नहीं आया।

Model proposed:
  काम → work
  अधूरा रह गया → remained unfinished
  थोड़ी मदद चाहिए → need a little help
  बहुत जल्दी → very quickly
  चला गया → left
  फ़र्क → difference
  समझ नहीं आया → didn't understand
  कल → tomorrow

Kept after validation and ranking:
  काम → work
  थोड़ी मदद चाहिए → need a little help
  बहुत जल्दी → very quickly
  फ़र्क → difference
  कल → tomorrow
  अधूरा रह गया → remained unfinished
  चला गया → left
  समझ नहीं आया → didn't understand
```

The loanword `मीटिंग` was not proposed at all — the prompt excluded it before the deterministic filter ran. Common words (`काम`, `मदद`, `जल्दी`, `फ़र्क`) were ranked first.

Kannada

```
Input:
  ನಾಳೆ ನನಗೆ ಮೀಟಿಂಗ್ ಇದೆ, ಆದರೆ ನಾನು ಆಫೀಸ್‌ಗೆ ಹೋಗಬೇಕು.
  ನನ್ನ ಕೆಲಸ ಮುಗಿಯಲಿಲ್ಲ, ಸ್ವಲ್ಪ ಸಹಾಯ ಬೇಕು.
  ಅವನು ತುಂಬಾ ಬೇಗ ಹೋದನು.

Kept:
  ನಾಳೆ → tomorrow
  ಸಹಾಯ ಬೇಕು → need help
  ಸ್ವಲ್ಪ → a little
  ತುಂಬಾ ಬೇಗ → very quickly
  ಕೆಲಸ → work
  ಆದರೆ → but
  ಮುಗಿಯಲಿಲ್ಲ → didn't finish
  ಹೋಗಬೇಕು → must go
```

Tamil

```
Input:
  நாளை எனக்கு மீட்டிங் இருக்கிறது, ஆனால் ஆபீஸ் போக வேண்டும்.
  என் வேலை முடியவில்லை, கொஞ்சம் உதவி வேண்டும்.
  அவன் மிகவும் சீக்கிரம் போனான்.

Kept:
  நாளை → tomorrow
  உதவி வேண்டும் → need help
  கொஞ்சம் → a little
  மிகவும் → very
  சீக்கிரம் → quickly
  போக வேண்டும் → have to go
  முடியவில்லை → could not
  இருக்கிறது → there is
  ஆனால் → but
  எனக்கு → to me
```

Telugu

```
Input:
  రేపు నాకు మీటింగ్ ఉంది, కానీ ఆఫీస్ కి వెళ్లాలి.
  నా పని పూర్తి కాలేదు, కొంచెం సహాయం కావాలి.
  అతను చాలా త్వరగా వెళ్ళాడు.

Kept:
  రేపు → tomorrow
  పని → work
  సహాయం → help
  త్వరగా → quickly
  వెళ్లాలి → have to go
  ఉంది → there is
  నా → my
  కావాలి → need
  పూర్తి కాలేదు → not finished
```

Deterministic filter, independent of the model

```
Input transcript:
  कल मीटिंग में काम की बात हुई और office भी गया

Proposed:
  मीटिंग में → in the meeting
  office     → workplace
  काम        → work
  बात        → matter

Kept:
  काम  → work
  बात  → matter
```

The loanword and the Latin-script word were dropped even though the model proposed them.

4.9 Test coverage

`backend/tests/test_language_bridge.py` contains **84 tests**. New coverage includes:

- a Latin-script `original` is dropped (parametrised across all four languages);
- a mixed-script span is dropped;
- an English loanword in the native script is dropped (parametrised across all four);
- a loanword with a postposition is still dropped;
- a common native word is kept (parametrised across all four);
- common words rank before rarer ones;
- every language has non-empty loanword, common-word and function-word sets;
- every list entry is written in the correct script (catches data typos);
- the prompt no longer contains the phrase "Harder words" and does contain concrete loanword examples for every language.

---

5. Part B — Sarvam speech recognition for Kannada, Tamil and Telugu

5.1 Model selection

Sarvam's REST endpoint exposes several models. All three candidates were tested against the project's key with the same audio:

Model	HTTP result	Decision
`saaras:v4`	200 OK	**Selected** — current recommended/latest model
`saaras:v3`	200 OK	Supported, previous generation
`saarika:v2.5`	200 OK	Supported, older

`saaras:v4` was chosen because it is Sarvam's current recommended model for Indic languages and supports the full range of output modes. The mode field is left blank by default, which uses the model's native-script transcription mode. It can be set with `SARVAM_STT_MODE` (for example `transcribe` or `codemix`).

5.2 Why Sarvam over Whisper for Indian languages

Whisper is a strong general multilingual model, and it remains the right choice for English and Hindi in this application. The case for Sarvam is narrower and specific: **for Kannada, Tamil and Telugu, an Indic-first model matches the language distribution of the task better than a general multilingual model does.**

The reasons, in order of importance for this project:

1. Training-data balance favours Indic-first models. Whisper is trained on a very large multilingual corpus, but that corpus is dominated by English and major European languages. Kannada, Tamil and Telugu sit in the long tail, so the effective number of hours and the coverage of their orthographies are much smaller. Sarvam's `saaras` family is trained with Indian languages as the core of the distribution rather than as a tail. Speech-recognition accuracy tracks in-domain data volume, and the effect is strongest for morphologically rich, lower-resource languages — exactly the three languages being routed.

2. Dravidian morphology and orthography are handled better by an Indic-first model. Kannada, Tamil and Telugu are agglutinative: a single whitespace token can carry a root plus several grammatical suffixes, and the written form involves complex vowel signs and consonant conjuncts. A model that has seen relatively little of that orthography tends to produce *phonetically plausible but orthographically wrong* spellings. That matters twice in this app: the transcript is shown to the learner, and the alternatives report quotes the transcript **verbatim** as the left-hand side of every item. Spelling noise therefore propagates into the vocabulary report and lowers the match rate. Sarvam's Indic-first training is intended to model those orthographies directly.

3. The language is an explicit, first-class input. Whisper's language handling is a prompt-level hint on top of auto-detection. In this project's earlier testing, auto-detection labelled Kannada as Tamil and Telugu as Hindi; the application had to pin the language to stop it. Pinning fixed the wrong-language failures, but it works around the model rather than using a model designed for the input. Sarvam accepts a BCP-47 code (`kn-IN`, `ta-IN`, `te-IN`) as a required field, so the language is part of the request contract rather than a guess that is sometimes corrected afterwards.

4. Code-mixed speech is an explicit design case. Everyday Indian speech mixes English words into the native language. Whisper has no output-mode control: depending on the audio it may keep English in Latin script, transliterate it into the native script, or translate it. Sarvam exposes explicit modes — `transcribe` (native script), `codemix` (mixed Indic/English), `verbatim`, `translit` and `translate` — so the behaviour can be chosen rather than hoped for. This project currently uses the default native-script mode; `codemix` is a documented option if mixed speech becomes the dominant case.

5. The project's own observations pointed the same way. The bridge feature was already built around the fact that Whisper's Kannada, Tamil and Telugu output carried more spelling noise than its Hindi output, and that its auto-detection confused the three languages. Sarvam returned coherent native-script output for `kn-IN` and accepted the model and language codes directly. That is consistent with the design argument above, but it is **not** a controlled accuracy measurement — see the claim boundary below.

6. Vendor focus matches product focus. Sarvam builds specifically for Indian languages and Indian speech use cases. For a product whose bridge mode is explicitly Indic, an Indic-first vendor is the more natural default than a general-purpose model in which those languages are a minority. This is a product-fit argument, not an accuracy number.

What was given up

The choice is a trade, and the costs are real:

Cost	Impact in this project	Why it is acceptable
Higher latency	~3.1 s for one short clip in a single observation	The full turn was ~7.6 s including LLM and TTS; the bridge is conversational, not real-time captioning
No word-level timestamps	`Transcription.words` is empty for Sarvam turns	Fluency analysis, the only consumer of word timings, is refused for bridge sessions
Paid credits	A second metered provider	The free Groq tier remains for English and Hindi; Sarvam is limited to three languages
A second vendor to operate	Two STT paths to monitor	Routing is centralised and `/health` reports both

Why Hindi stays on Whisper

The same argument does **not** apply to Hindi with the same force. Whisper is very strong on Hindi, Hindi has far more training data than the three Dravidian languages, and the project's bridge tests and prompts were built and validated around the Whisper/Hindi path. Moving Hindi would add risk without a demonstrated gain. The routing layer makes that decision reversible: adding `hi` to `SARVAM_STT_LANGUAGES` is a configuration change, not a code change, once a benchmark justifies it.

Claim boundary

This section states a **design rationale**, not a measured word-error-rate improvement. The project did not run a controlled same-audio benchmark against Whisper, so no accuracy percentage is claimed. A WER/CER study on real learner recordings is listed as future work (§8). The reasons above are the ones that justified the switch; the benchmark is what would quantify it.

5.3 API integration

Sarvam's transcription API differs from the OpenAI-compatible shape used by Groq:

Aspect	Groq / Whisper	Sarvam
Endpoint	`POST /openai/v1/audio/transcriptions`	`POST /speech-to-text`
Authentication	`Authorization: Bearer …`	`api-subscription-key: …`
Body	multipart `file`, `model`, `language`	multipart `file`, `model`, `language_code`
Language code	ISO 639-1 (`kn`)	BCP-47 (`kn-IN`)
Word timings	Yes (`timestamp_granularities[]=word`)	No (sentence/chunk timestamps only)
Response	`text`, `words[]`	`transcript`, `language_code`, `request_id`

The adapter maps ISO codes to BCP-47 (`kn` → `kn-IN`). Because Sarvam returns no word timings, `Transcription.words` is empty for those turns. This is acceptable in the current design: the only consumer of word timings is the fluency analysis, which is refused for bridge sessions.

5.4 Error translation

Provider-specific failures are translated into application errors in the client, so no HTTP or Sarvam-shaped payload reaches the service layer:

Provider response	Application error
401 / 403	`CONFIG_ERROR` — the Sarvam key was rejected
402	`CONFIG_ERROR` — no remaining credit
400 / 404 / 422	`CONFIG_ERROR` — usually a bad model ID or disabled language code
429	`PROVIDER_RATE_LIMITED`, preserving `Retry-After`
5xx	`PROVIDER_UNAVAILABLE`
Timeout / transport failure	`PROVIDER_TIMEOUT` / `PROVIDER_UNAVAILABLE`

This matches the contract already used by the Groq and Deepgram clients.

5.5 Language routing

A routing adapter selects the provider per turn:

```
class LanguageRoutingSpeechToTextProvider:
    def provider_for(self, language: str | None) -> SpeechToTextProvider:
        if not language:
            return self._default
        return self._by_language.get(language.strip().lower(), self._default)

    async def transcribe(self, audio, filename, mime_type, language=None):
        return await self.provider_for(language).transcribe(
            audio, filename, mime_type, language=language
        )
```

The registry builds the router only when all of the following hold:

1. the default STT provider is not `mock`;
2. `SARVAM_STT_LANGUAGES` is non-empty;
3. `SARVAM_API_KEY` is set.

If the key is missing, the three languages keep working on Whisper and a warning is logged at startup. This avoids turning a missing optional key into a broken conversation path.

Default routing:

Spoken language	Provider	Model
English (`None` language)	Groq	`whisper-large-v3-turbo`
Hindi (`hi`)	Groq	`whisper-large-v3`
Kannada (`kn`)	Sarvam	`saaras:v4`
Tamil (`ta`)	Sarvam	`saaras:v4`
Telugu (`te`)	Sarvam	`saaras:v4`

Hindi deliberately remains on Whisper: it was already strong there, and keeping the incumbent avoids changing behaviour for the language that was tested most.

5.6 Health reporting

`GET /health` now reports the active routes:

```
"providers": {
  "stt": "groq",
  "stt_language_routes": {"kn": "sarvam", "ta": "sarvam", "te": "sarvam"},
  ...
}
```

`GET /health/providers` checks every provider the router can reach, once each, so a failing Sarvam configuration cannot hide behind a healthy default. Sarvam has no public model-list endpoint, so its check reports `model_listed: null` ("not checked"); the health route treats that as healthy and the model is verified on first use.

5.7 End-to-end verification

A Kannada bridge session was created and a WAV file posted to the turn endpoint. The backend log shows the full chain:

```
POST https://api.sarvam.ai/speech-to-text            200 OK
POST https://api.groq.com/openai/v1/chat/completions 200 OK
POST https://api.deepgram.com/v1/speak               200 OK
```

Observed turn:

```
Transcript: ಹಲೋ, ಐ ಗೋ ಟು ಕಾಲೇಜ್ ಯೆಸ್ಟರ್ಡೇ ಅಂಡ್ ಇಟ್ ವಾಸ್ ವೆರಿ ಗುಡ್...
Reply:      That sounds like a great day at college. What was the most
            interesting part of the lesson for you?
Timings:    stt 3154 ms | llm 1229 ms | tts 3216 ms | total 7599 ms
```

The audio used was an English sample, so the transcript is English phonetics written in Kannada script. That is the expected result of pinning `kn-IN`; it verifies routing and formatting, not Kannada accuracy.

5.8 Whisper versus Sarvam: a qualitative comparison

The two engines trade different strengths. This table is based on their design and observed behaviour, not on a controlled word-error-rate experiment (see §7).

Dimension	Whisper on Groq	Sarvam `saaras:v4`
Kannada / Tamil / Telugu accuracy	General multilingual model; more spelling noise	Purpose-built for Indic languages; generally stronger
Hindi accuracy	Very strong	Good, not clearly better
English accuracy	Very strong	Not the target
Latency	Groq specialises in fast inference	REST call, roughly 3 s for a short clip in one observation
Word-level timestamps	Yes	No
Code-mixed English	Often preserved in Latin script	`transcribe` mode writes it in native script; `codemix` mode is available
Cost	Generous free tier	Paid credits

The switch is a deliberate trade: **better native-language transcription in exchange for speed and word timings.** Neither loss affects the language-bridge feature today.

---

6. Evaluation

6.1 Automated tests

```
442 passed
```

Breakdown of the directly relevant suites:

Suite	Tests	Focus
`tests/test_language_bridge.py`	84	Bridge boundaries, alternatives filtering, storage, API
`tests/test_sarvam_stt.py`	13	Sarvam adapter, error translation, routing, registry fallback
Full suite	442	No regressions elsewhere

The Sarvam tests use `respx` to intercept HTTP, so they assert the exact request shape (header name, multipart fields, BCP-47 language code) without spending credits.

6.2 Live verification

Check	Result
Sarvam key accepted	Yes
`saaras:v4`, `saaras:v3`, `saarika:v2.5` reachable	Yes (all HTTP 200)
Kannada turn routed to Sarvam	Yes — confirmed in backend log and transcript
Hindi turn routed to Whisper	Yes — unchanged path
Alternatives report on native transcripts	Native words kept; loanwords and Latin-script words excluded
`/health` reports routes	Yes — `{"kn": "sarvam", "ta": "sarvam", "te": "sarvam"}`

6.3 Performance

One short Kannada-pinned clip produced an STT time of **3,154 ms**. This is a single observation on a REST endpoint, not a benchmark. It is slower than the Groq path but small relative to the full turn (7,599 ms including LLM and TTS).

6.4 Threats to validity

- **No controlled WER/CER benchmark.** The report does not claim a measured accuracy improvement for Sarvam. The vendor choice is justified by design intent and the project's own observation of Whisper's spelling noise, not by a published experiment.
- **Hand-written evaluation transcripts.** The alternatives examples in §4.8 were passed directly to the provider to isolate prompt and filter behaviour. They were not spoken by learners.

- **One integration audio sample.** The end-to-end test used an English clip pinned to a Kannada session. It validates routing, not transcription quality.
- **Single-run model output.** Language-model output varies slightly between runs; the filters, not the examples, are what the tests pin down.

---

7. Limitations

1. **Context-dependent words can be mistranslated.** In the Hindi example, `कल` was translated as *tomorrow* although the sentence implies *yesterday*. Word-level translation without an explicit context instruction cannot always disambiguate. A future prompt revision should tell the model to choose the sense that fits the sentence.
2. **The loanword and common-word lists are finite and curated.** They cover high-frequency vocabulary, not dialects, slang or domain terms. They need occasional maintenance as recognition output evolves.
3. **Code-mixed speech may read oddly on Sarvam.** In the default `transcribe` mode, English words are written in the native script. This is harmless for the vocabulary report (the loanword filter removes them) but can make the raw transcript less readable. `codemix` mode is available but has not been evaluated.
4. **No word timings from Sarvam.** Acceptable now because fluency analysis is refused for bridge sessions. If fluency is ever extended to bridge languages, a timing-capable provider or an alignment step will be needed.
5. **Two STT vendors to operate.** Hindi and English depend on Groq; the other three depend on Sarvam. A failure in either affects a subset of languages, and monitoring must cover both.
6. **The report is unscored.** It deliberately says nothing about whether an utterance was good; that remains the job of the English analyses.

---

8. Future work

1. **Run a controlled WER/CER benchmark** on real learner recordings per language, with identical audio sent to Whisper and Sarvam. Report word error rate, character error rate and proper-noun handling.
2. **Evaluate `codemix` mode** for Hinglish and comparable code-mixed speech, and choose the mode per language or per turn.
3. **Add context disambiguation** to the prompt for words such as `कल` (yesterday/tomorrow), `समय` (time), and similar polysemous items.
4. **Confidence-based routing.** If a provider exposes confidence, fall back to the other engine on low-confidence segments.
5. **Grow the lexical data from corpora**, and add regression tests from real anonymised transcripts.
6. **Consider Sarvam for Hindi** only after a benchmark shows a gain; the current split is intentional.
7. **Surface the routing in the UI** (for example, a small "transcribed by" note) if operational transparency becomes useful.

9. Conclusion

The language bridge now does what it was meant to do: it takes words the learner actually said in their own language and shows them the English they could use next time. The fix works at two levels — a prompt that asks for the right thing, and a validation layer that enforces the invariant regardless of what the model returns. The same logic covers all four languages, with per-language lexical data doing the language-specific work.

Kannada, Tamil and Telugu now use a speech-to-text engine built for Indian languages, routed per turn and degrading safely to the existing provider when unconfigured. Hindi and English keep the engine that already served them well.

The system remains fully tested (442 passing tests), operationally simple (no migration, no schema change), and reversible through configuration alone.

Appendix A — Configuration

```
# Sarvam speech-to-text for Kannada, Tamil and Telugu.
SARVAM_API_KEY=
SARVAM_BASE_URL=https://api.sarvam.ai
SARVAM_STT_MODEL=saaras:v4
# Blank uses the model's default (native-script transcription).
SARVAM_STT_MODE=
# ISO 639-1 codes routed to Sarvam. Empty puts every language back on Groq.
SARVAM_STT_LANGUAGES=kn,ta,te
```

To revert the routing without a code change, set `SARVAM_STT_LANGUAGES=` and restart the backend. Without a key, the listed languages already fall back to Whisper automatically.

Appendix B — Files changed or added

File	Role
`backend/app/languages.py`	Language metadata: script ranges, loanwords, common words, function words
`backend/app/analysis/alternatives.py`	Validation, loanword filtering, ranking, cap
`backend/app/providers/groq/alternatives.py`	Rewritten prompt and per-language prompt assembly
`backend/app/providers/sarvam/client.py`	Sarvam HTTP client and error translation
`backend/app/providers/sarvam/stt.py`	Sarvam speech-to-text adapter
`backend/app/providers/routing.py`	Per-language STT routing
`backend/app/providers/registry.py`	Builds the router; safe fallback when unconfigured
`backend/app/config.py`	Sarvam settings and derived route map
`backend/app/api/routes/health.py`	Exposes active STT routes
`backend/app/main.py`	Startup warnings for unconfigured Sarvam
`backend/tests/test_language_bridge.py`	84 bridge and alternatives tests
`backend/tests/test_sarvam_stt.py`	13 Sarvam and routing tests
`frontend/src/routes/alternatives.tsx`	Report copy updated
`frontend/src/routes/index.tsx`	Home-page bridge copy updated
`backend/README.md`, `DEPLOY.md`, `render.yaml`, `.env.example`	Documentation and deployment configuration

Appendix C — Reproducing the results

```
cd backend

# Full suite
uv run pytest -q

# The two directly relevant suites
uv run pytest tests/test_language_bridge.py tests/test_sarvam_stt.py -q

# Provider health (requires the backend to be running)
# http://localhost:8001/health
# http://localhost:8001/health/providers
```

References

1. Sarvam AI, *Speech to Text API* — `https://docs.sarvam.ai/api-reference-docs/speech-to-text/transcribe`
2. Groq, *Speech to Text* (Whisper large-v3 / turbo) — `https://console.groq.com/docs/speech-to-text`
3. A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, I. Sutskever, "Robust Speech Recognition via Large-Scale Weak Supervision," *ICML*, 2023.
4. Unicode Consortium, *The Unicode Standard — Devanagari, Kannada, Tamil, Telugu blocks*.
5. Deepgram, *Text to Speech models* — `https://developers.deepgram.com/docs/tts-models`
6. FastAPI documentation — `https://fastapi.tiangolo.com/`